Alternative Assessment Tool (AAT) Report on

"Simulation of Combinational and Sequential Modules in Cadence Tool"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23EC5PE1AD]

Submitted by

Shashwath S Parol 1BM24EC191

Link to GitHub

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

1. 4-Bit Parity Generator and Checker

PROCEDURE:

1. Open the terminal and navigate to the directory where the Verilog design and testbench files are stored.

2. Open Gedit and write the Verilog code for the 4-bit parity generator and checker. Save the design file with a .v extension.

3. Open another Gedit window and write the testbench code to apply different input combinations to the design. Save the testbench file with a .v extension.

4. Open the terminal and compile the Verilog design and testbench using the Cadence Verilog compiler.

5. Execute the compiled simulation using the appropriate simulation command.

6. After successful compilation and simulation, open SimVision to view the simulation waveforms.

7. In SimVision, select the required input and output signals from the design hierarchy and add them to the waveform window.

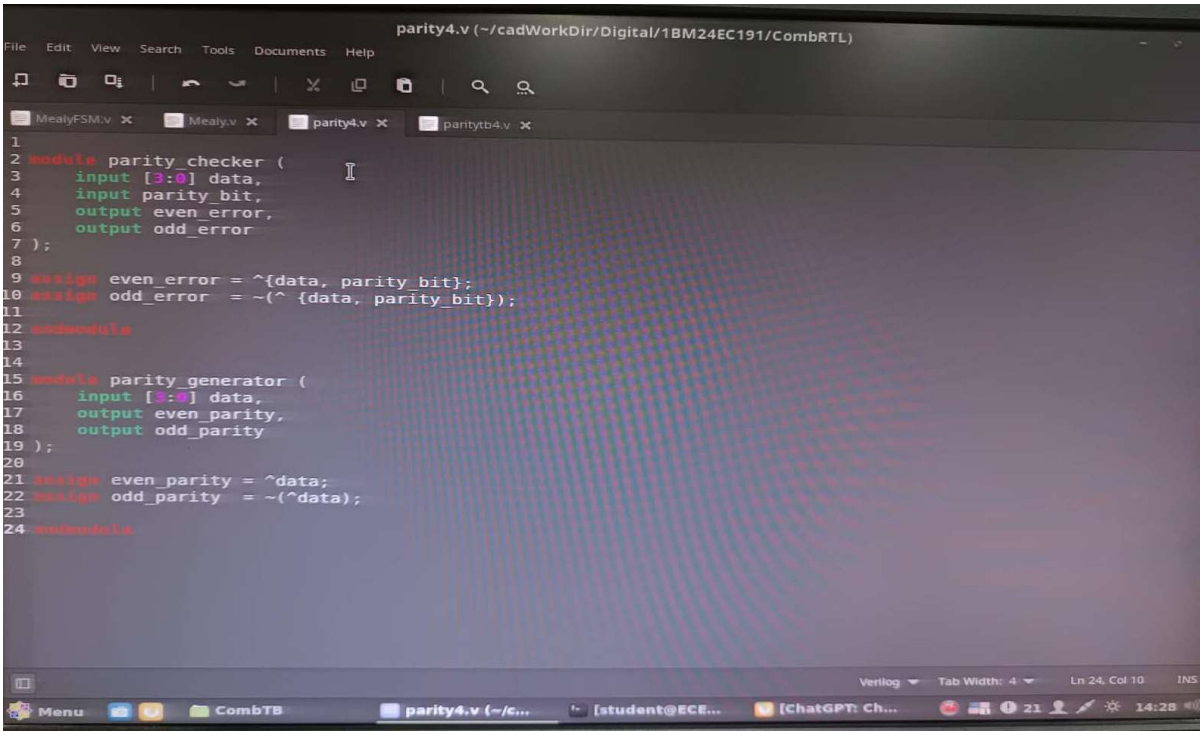
8. Run the simulation and observe the changes in the input and output signals for different input combinations.

9. Verify that the even parity generator produces a parity bit such that the total number of 1s is even, and the odd parity generator produces a parity bit such that the total number of 1s is odd.

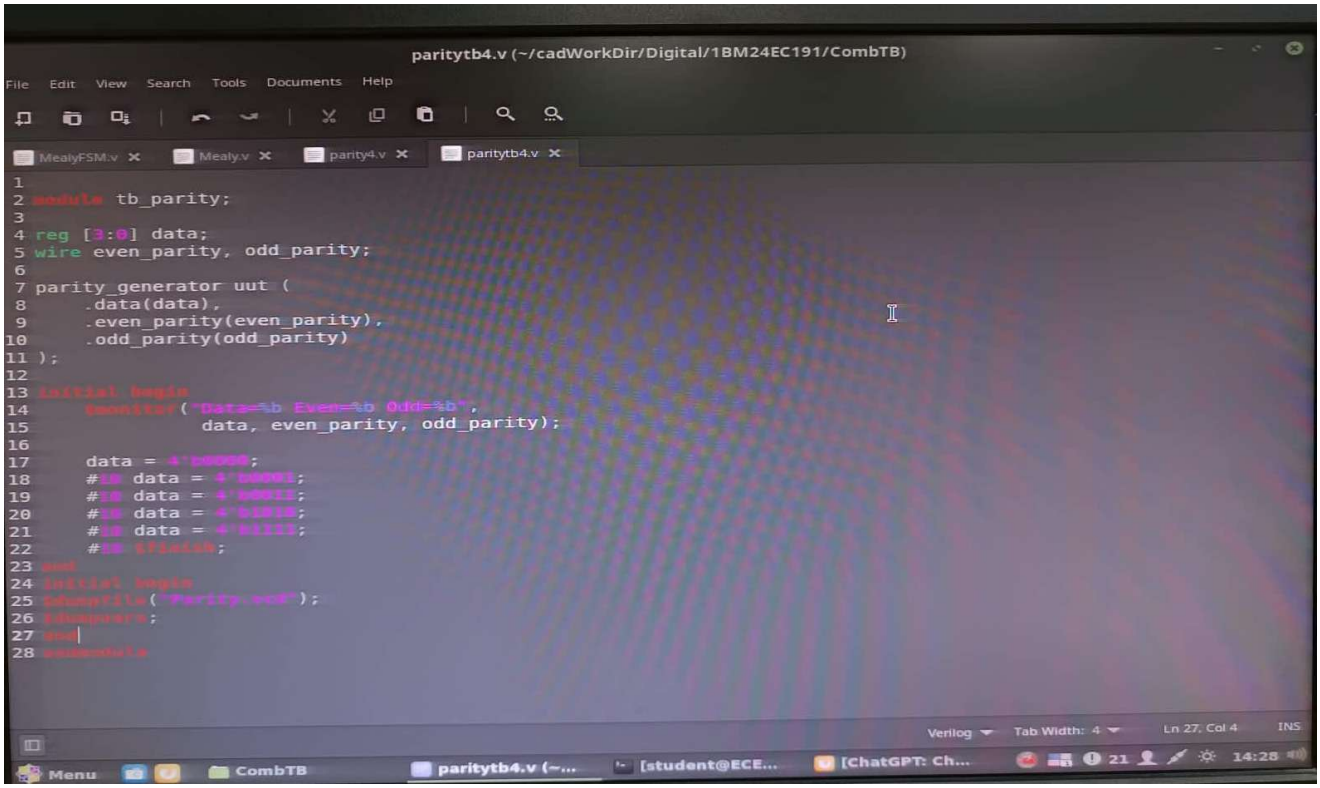
10. Verify that the parity checker detects errors when the received data or parity bit is altered.

11. Observe and capture the simulation waveforms for the laboratory record.

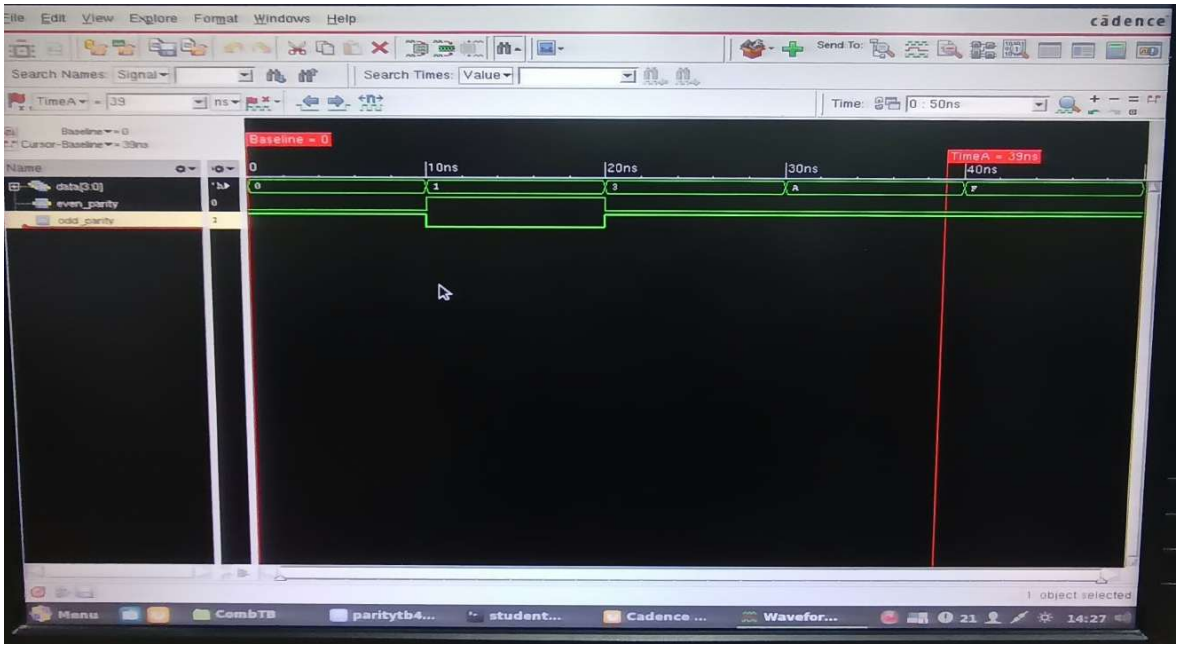
CODE:



TESTBENCH:



OUTPUT:



RESULT:

The 4-bit parity generator and checker were successfully simulated using Cadence tools. The output waveforms were observed in SimVision and verified for different input combinations. The parity generator produced the correct even and odd parity bits, and the parity checker detected errors in the received data.

2. Mealy FSM 101 Detector

PROCEDURE:

1. Open the terminal and navigate to the directory where the Verilog design and testbench files are stored.

2. Open Gedit and write the Verilog code for the Mealy sequence detector to detect the sequence 101. Save the design file with a .v extension.

3. Open another Gedit window and write the testbench code to apply different input sequences to the design. Save the testbench file with a .v extension.

4. Open the terminal and compile the Verilog design and testbench using the Cadence Verilog compiler.

5. Execute the compiled simulation using the appropriate simulation command.

6. After successful compilation and simulation, open SimVision to view the simulation waveforms.

7. In SimVision, select the required input and output signals from the design hierarchy and add them to the waveform window.

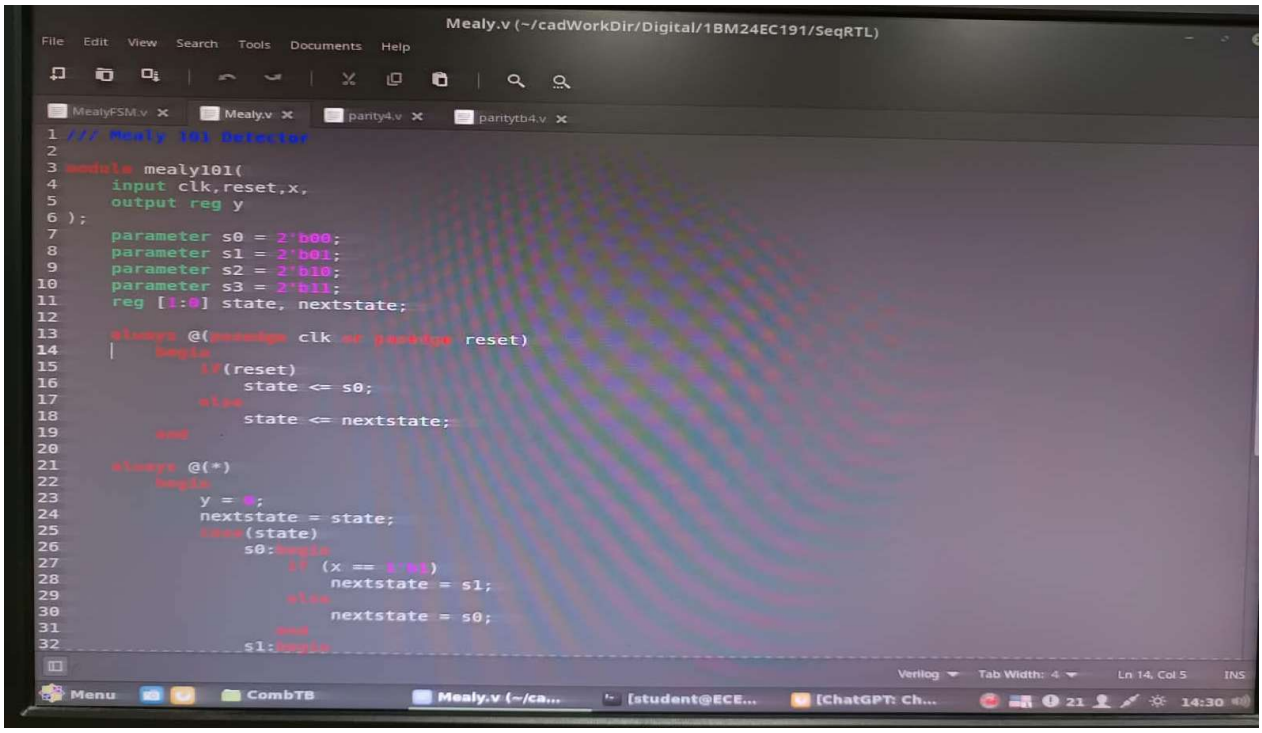
8. Run the simulation and observe the changes in the input and output signals for different input sequences.

9. Verify that the output becomes HIGH when the input sequence 101 is detected.

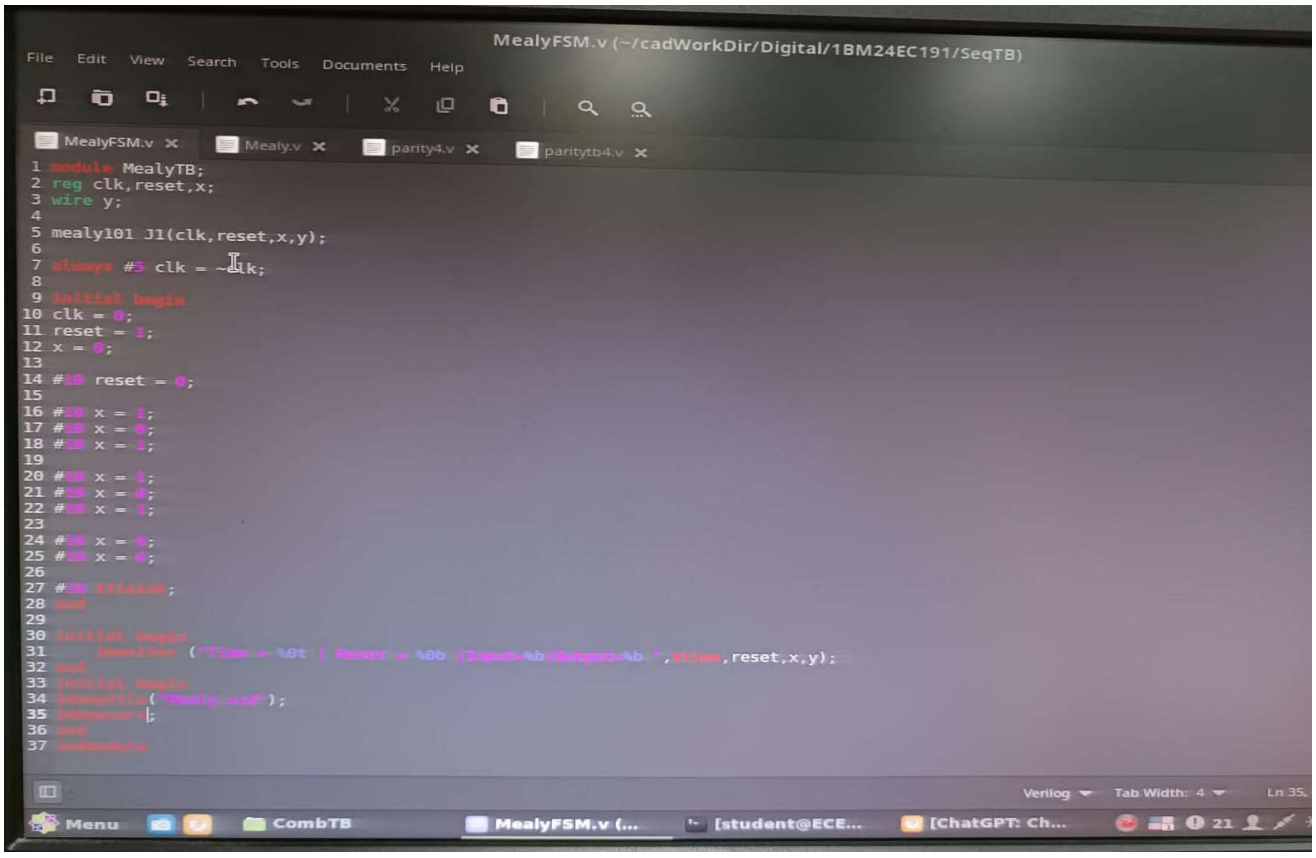
10. Apply different input sequences and verify the operation of the Mealy sequence detector, including overlapping sequence detection.

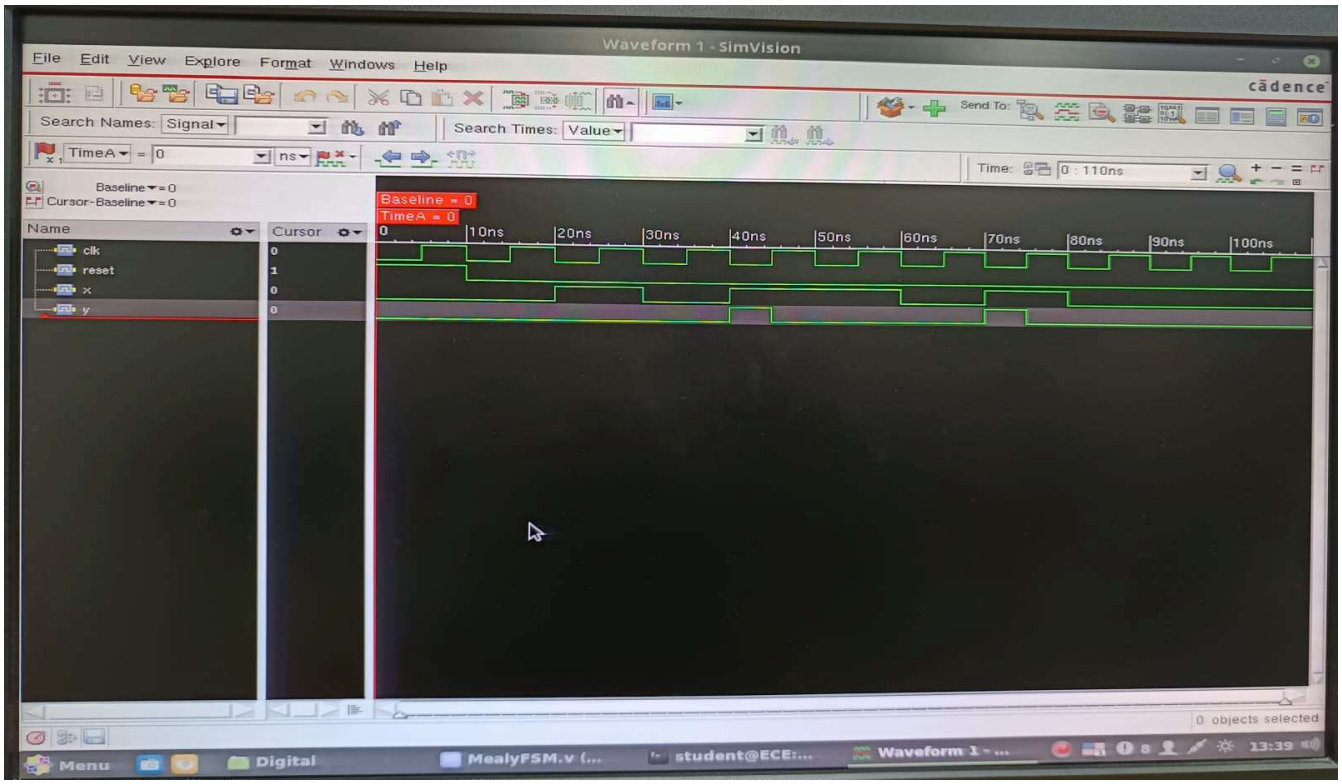
11. Observe and capture the simulation waveforms for the laboratory record.

CODE:



TESTBENCH:





RESULT:

The Mealy 101 sequence detector was successfully simulated using Cadence tools. The output waveforms were observed in SimVision and verified for different input sequences. The detector produced the output HIGH when the sequence 101 was detected, confirming the correct operation of the Mealy state machine.